Terraform Day 01

Infrastructure as Code Fundamentals

Session Details

- **Duration:** 150 minutes (2.5 hours)
- **Breaks:** 10 min at 60 min, 10 min at 120 min
- **Format:** Interactive lecture with hands-on

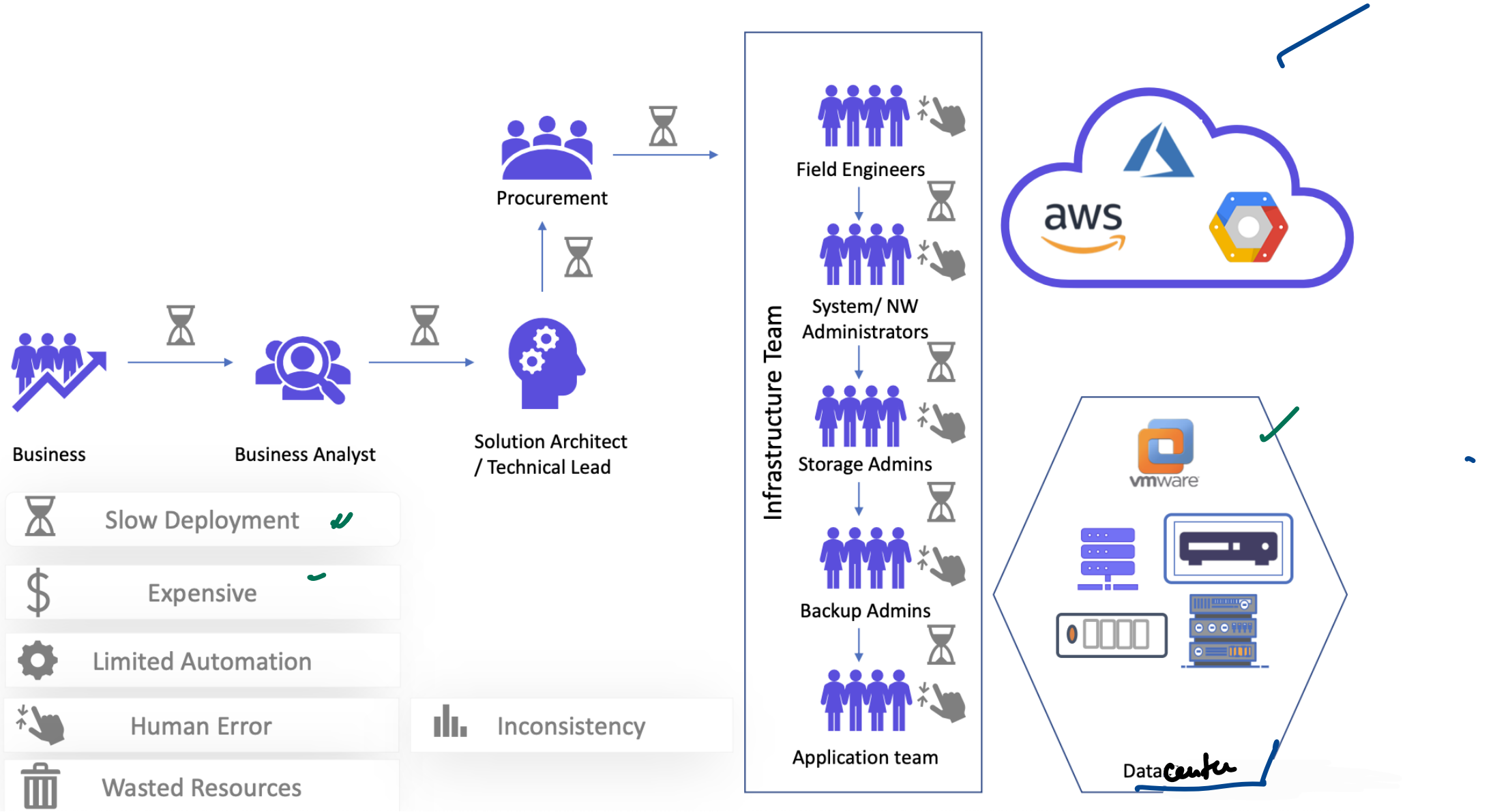
Today's Journey

- Manual Deployment Challenges (15 min)
- Evolution of Infrastructure Management (20 min)
- Types of IaC Tools (25 min)
- Why Terraform? (20 min)
- Terraform Core Concepts (25 min)
- HCL Basics & First Resource (20 min)

Manually

Different challenges?

1. Time consuming
2. No versioning
3. Painful
4. Lack of consistency
5. No collaboration
6. Reversing is hard
7. DR is complicated



Courtesy: Google image

Types of IaC Tools Overview

Three Categories of IaC Tools

1. Configuration Management

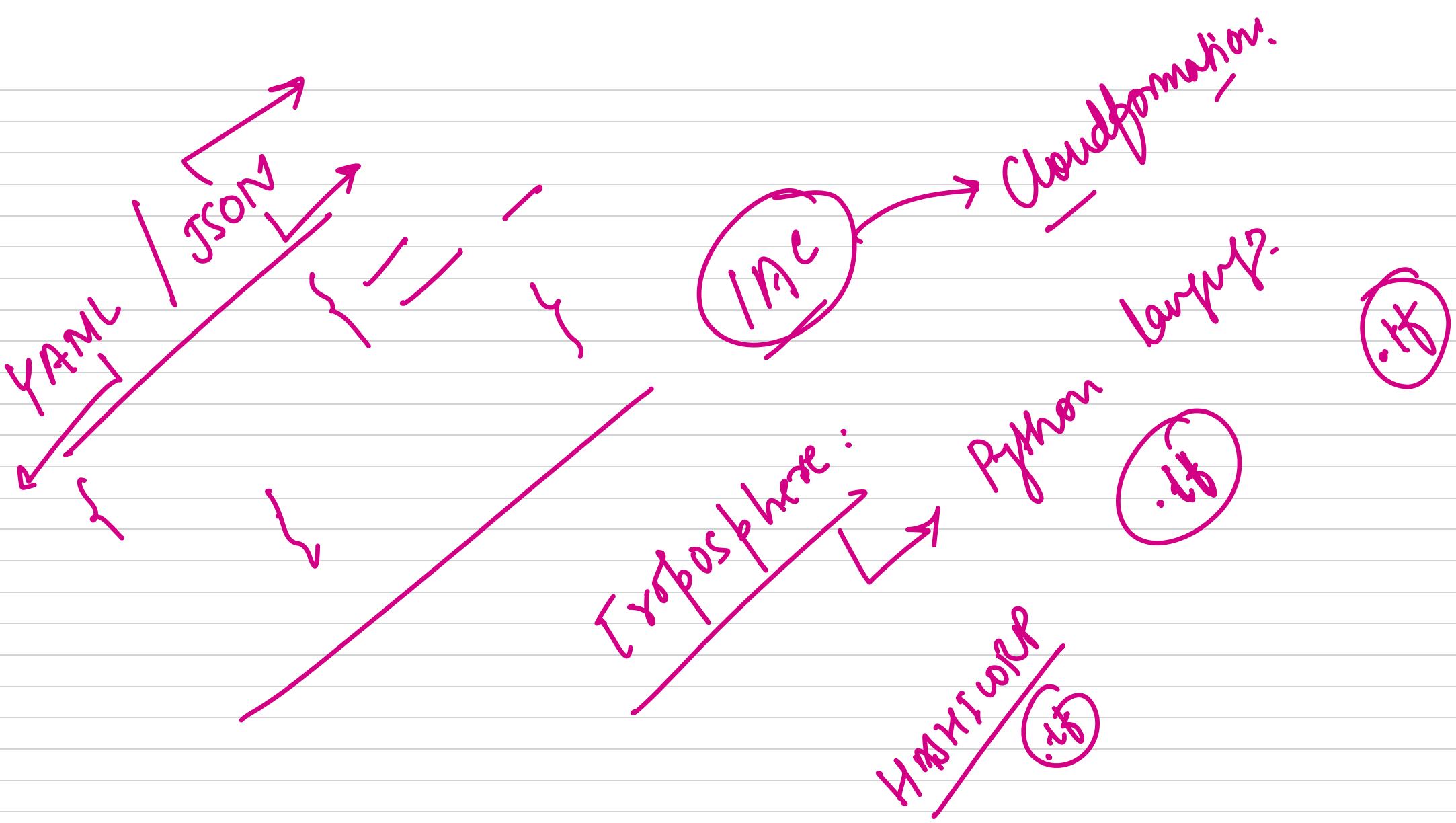
- **Ansible** - Agentless, YAML-based
- **Puppet** - Agent-based, declarative
- **Chef** - Agent-based, Ruby DSL
- **SaltStack** - Agent/agentless, Python-based

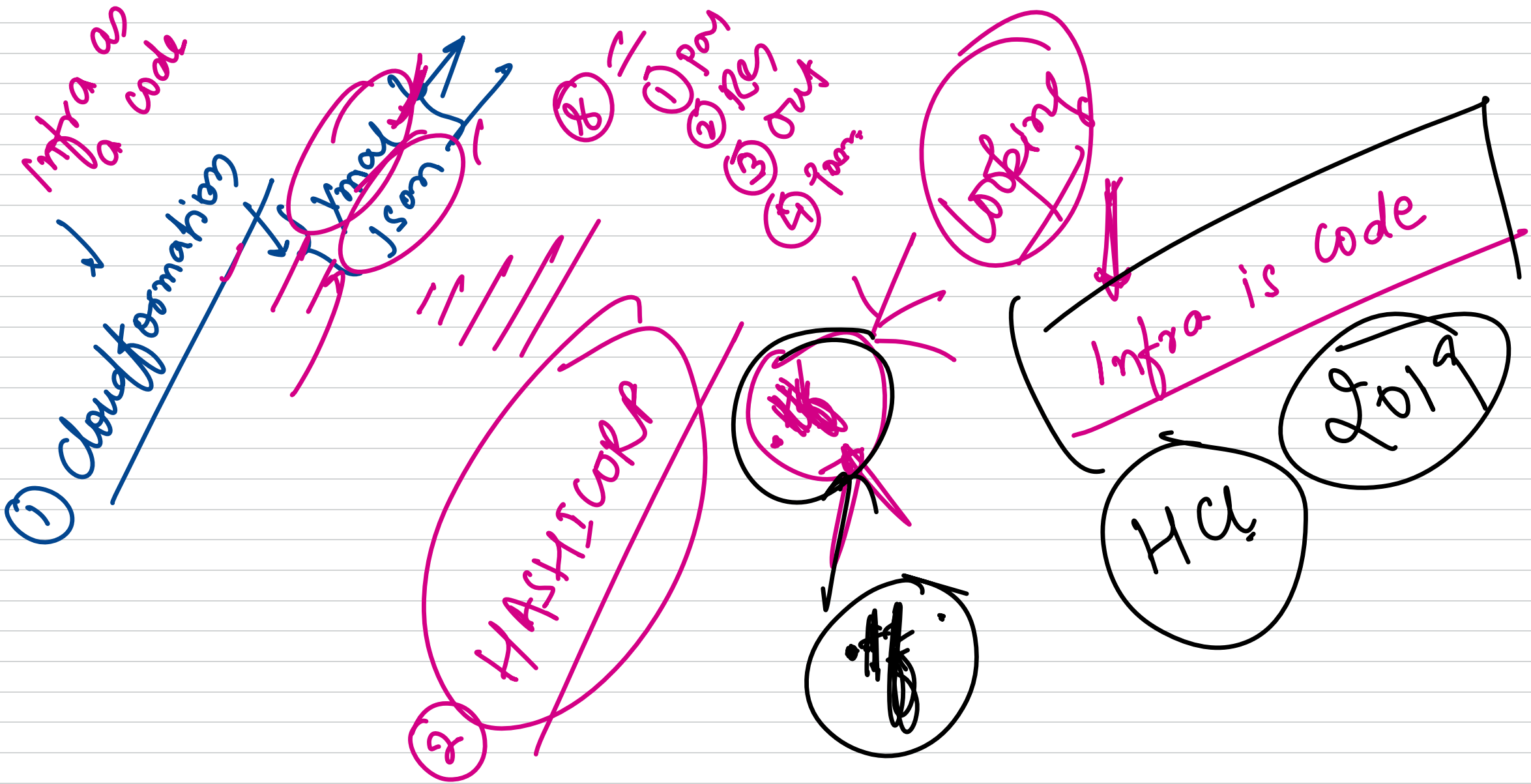
2. Server Templating

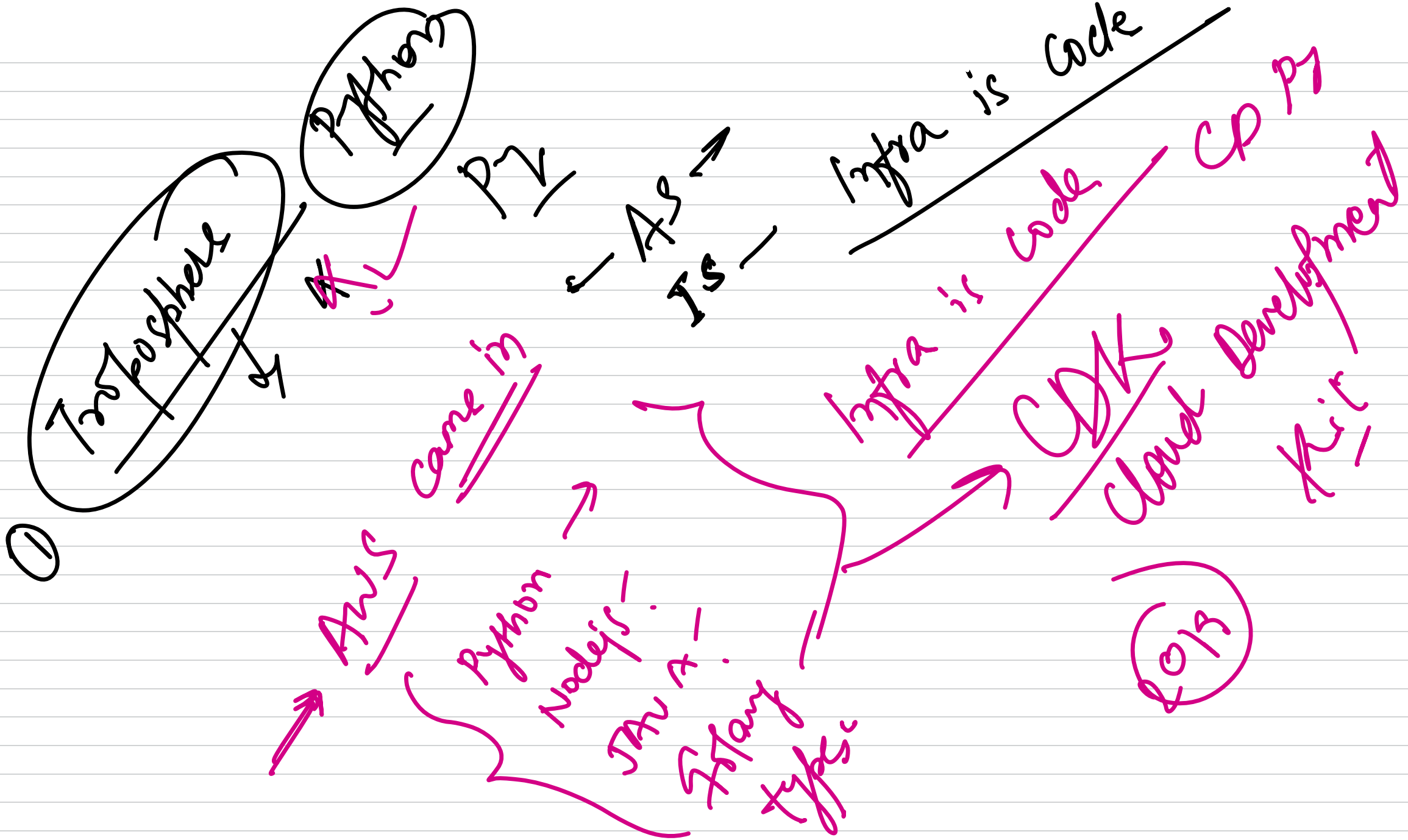
- **Docker** - Container images
- **Packer** - VM/AMI images
- **Vagrant** - Development environments

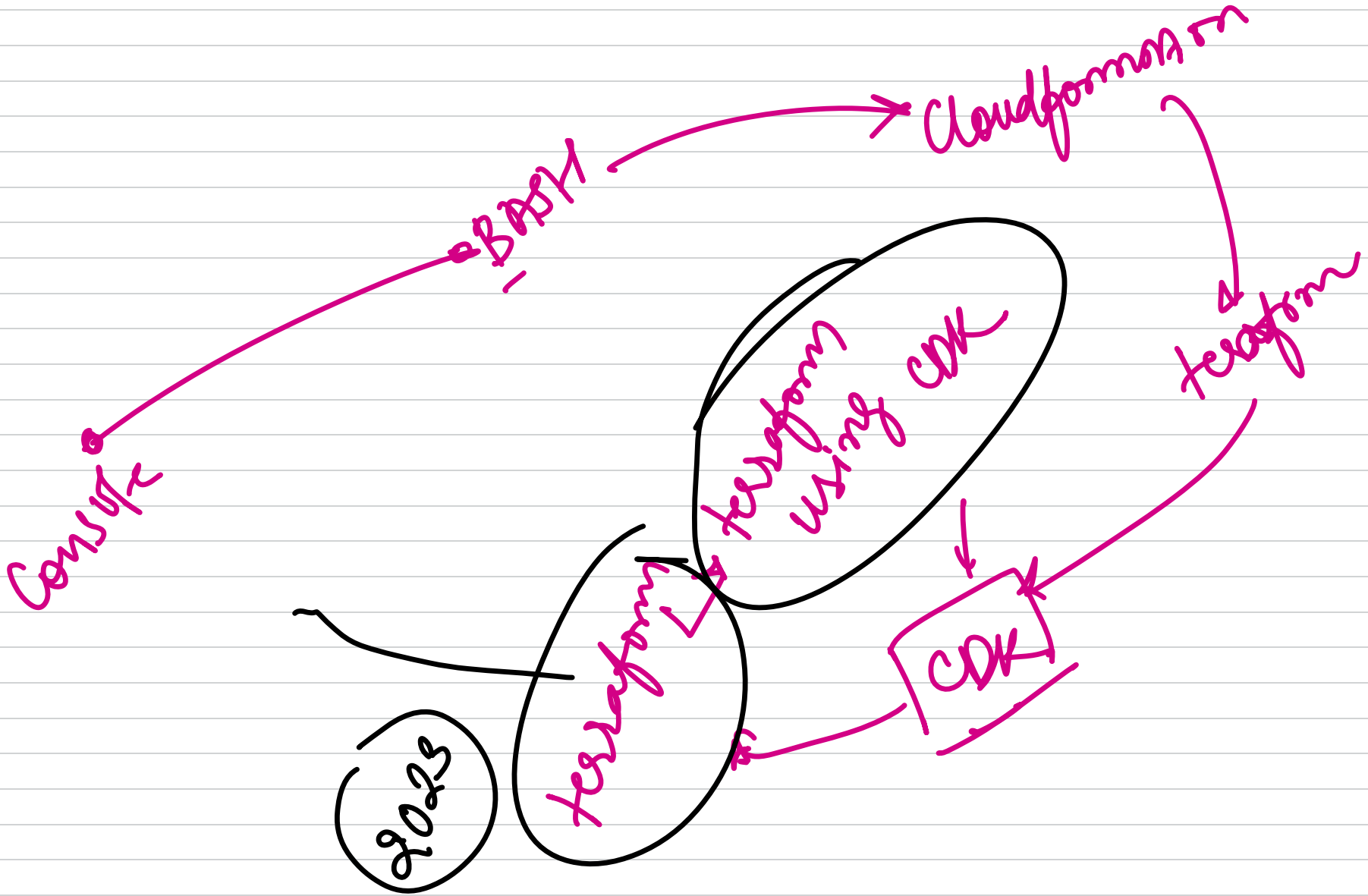
3. Provisioning Tools

- **Terraform** - Multi-cloud provisioning
- **CloudFormation** - AWS-specific
- **Pulumi** - Programming language-based


YAML / JSON
IAC
Cloudformation.
Troposphere :
Python
language.
.tf
.tf
.tf


Infra as a code
① Cloudformation
Yaml
Json
② HASHICORP
① Plan
② Res
③ Out
④ Para
Outputs
Infra is code
2019
HCL


Infrastructure
Python
API
Infra is Code
AWS came in
Python
Nodejs
JSII
Golang
Typescript
Infra is Code
CDK
Cloud development kit
CDK
2019


Cloudformation
BASH
Terraform using CDK
Terraform
2023
CDK
Terraform

Configuration Management Deep Dive

Configuration Management Tools

Purpose: Install and manage software on existing servers

Key Characteristics

- **Standard Structure** - Consistent server configurations
- **Version Control** - Track configuration changes
- **Idempotent** - Safe to run multiple times
- **Agent-based/Agentless** - Different deployment models

Example: Ansible Playbook

```
- name: Install web server
  hosts: webservers
  tasks:
    - name: Install Apache
      yum: name=httpd state=present
    - name: Start Apache
      service: name=httpd state=started
```

Server Templating Tools

Server Templating Philosophy

Purpose: Pre-install software and dependencies into images

Key Benefits

- **Immutable Infrastructure** - Replace, don't modify
- **Faster Deployments** - Pre-built images
- **Consistency** - Same image across environments
- **Rollback Capability** - Previous image versions

Examples

- **Docker:** Container images with applications
- **Packer:** VM images (AMIs, VMDKs)
- **Vagrant:** Development environment boxes

Immutable vs Mutable

```
Mutable:
Server → Configure → Update → Patch

Immutable:
Image → Deploy → Replace → Deploy New Image
```

Provisioning Tools

Provisioning Tools Purpose

Deploy immutable infrastructure resources

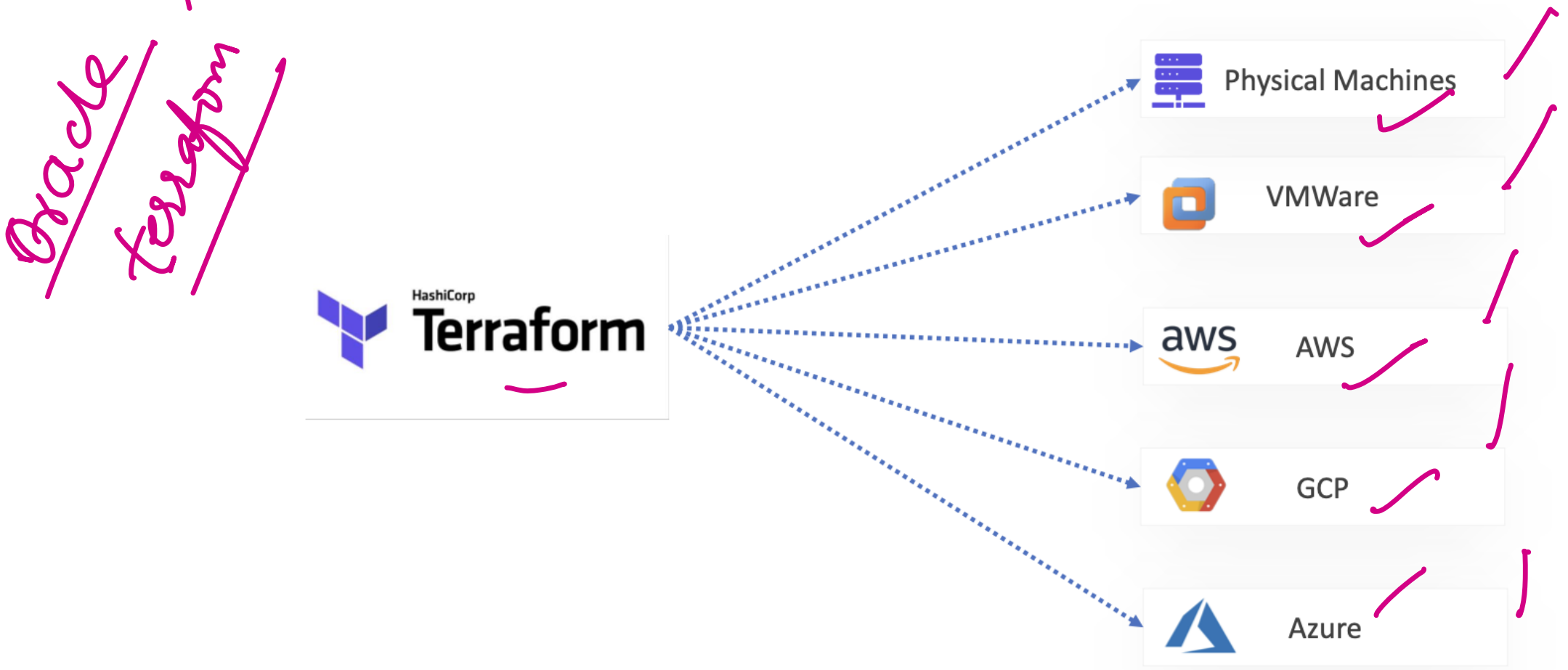
What They Provision

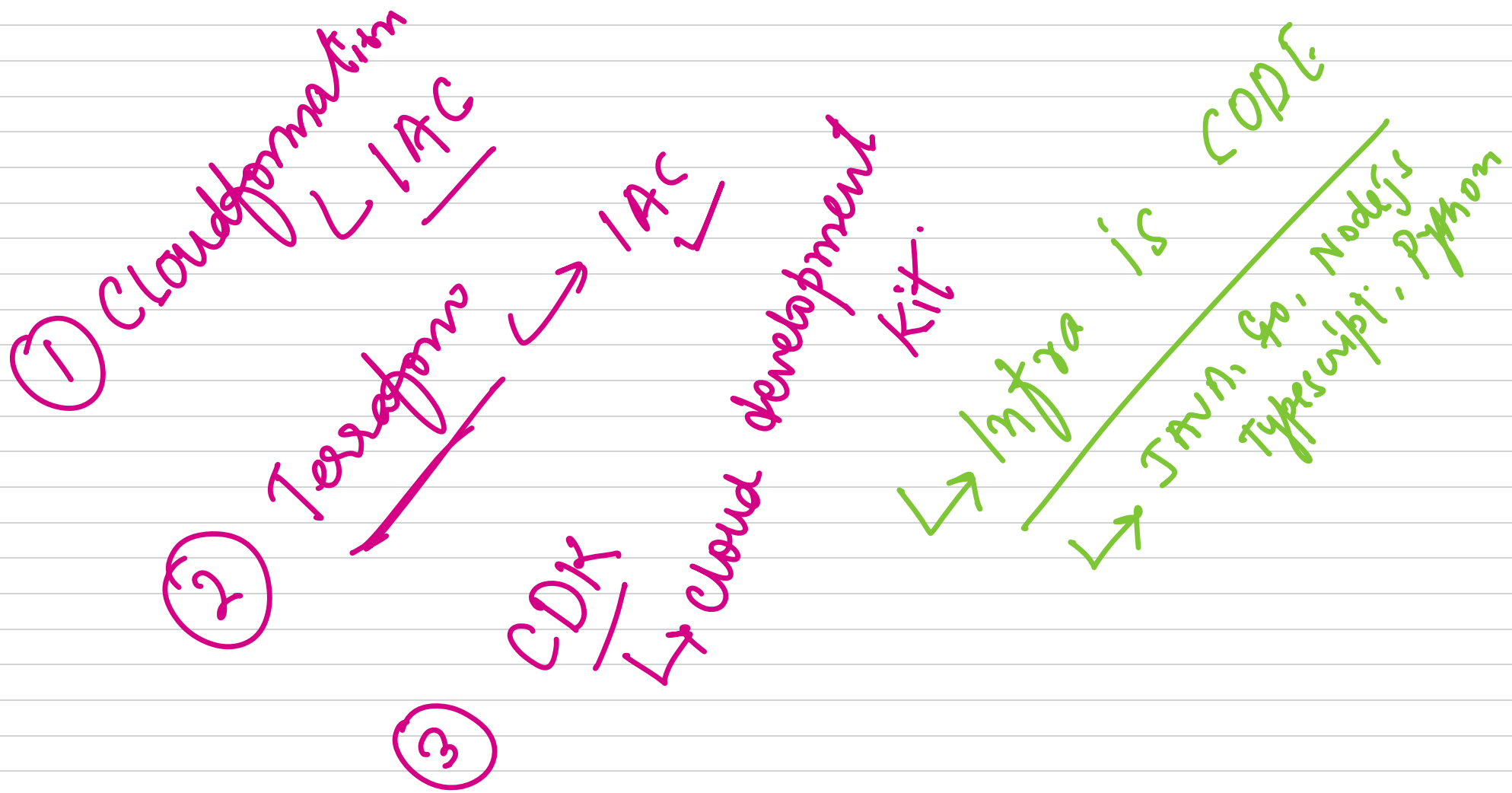
- **Servers** - EC2, VMs, Containers
- **Databases** - RDS, DynamoDB, MongoDB
- **Network Components** - VPCs, Subnets, Load Balancers
- **Security** - IAM, Security Groups, Policies

Multi-Provider Support

- **Cloud Providers** - AWS, Azure, GCP
- **On-Premises** - VMware, OpenStack
- **SaaS** - GitHub, DataDog, Auth0

Why Terraform?




① Cloudformation
↳ IAC
② Terraform
↳ IAC
③ CDK
↳ cloud development kit
↳ Infra is CODE
↳ Java, Go, Nodejs
Typescript, Python

Why Terraform?

Terraform's Unique Advantages

1. Multi-Platform Support

- **Physical Machines** - Bare metal servers
- **Virtualization** - VMware vSphere
- **Cloud Providers** - AWS, Azure, GCP
- **Hybrid/Multi-Cloud** - Consistent tooling

2. Massive Provider Ecosystem

Infrastructure Providers:

AWS, Azure, GCP, Alibaba Cloud

VMware, OpenStack, Kubernetes

Monitoring & Security:

DataDog, Grafana, Sumo Logic

Palo Alto, Auth0, Vault

Databases & Storage:

MongoDB, MySQL, PostgreSQL

InfluxDB, Elasticsearch


Infrastructure
AWS
Azure
GCP
VMware
Monitoring
DataDog
Grafana
Wavefront
Sumo Logic
Security
Auth0
Palo Alto
Vault
Okta
Networking
CloudFlare
BigIP
DNS
Infoblox
Databases
MongoDB
MySQL
PostgreSQL
InfluxDB
Version Control
GitHub
GitLab
Bitbucket
Azure DevOps

The IaC Revolution

- **Declarative** - Describe what you want, not how
- **Version Controlled** - Git for infrastructure
- **Repeatable** - Same result every time
- **Collaborative** - Team-based infrastructure
- **Auditable** - Track all changes

Configuration Management Deep Dive

Configuration Management Tools

Purpose: Install and manage software on existing servers

Key Characteristics

- **Standard Structure** - Consistent server configurations
- **Version Control** - Track configuration changes
- **Idempotent** - Safe to run multiple times
- **Agent-based/Agentless** - Different deployment models

Example: Ansible Playbook

```
- name: Install web server
  hosts: webservers
  tasks:
    - name: Install Apache
      yum: name=httpd state=present
    - name: Start Apache
      service: name=httpd state=started
```

Server Templating Tools

Server Templating Philosophy

Purpose: Pre-install software and dependencies into images

Key Benefits

- **Immutable Infrastructure** - Replace, don't modify
- **Faster Deployments** - Pre-built images
- **Consistency** - Same image across environments
- **Rollback Capability** - Previous image versions

Examples

- **Docker:** Container images with applications
- **Packer:** VM images (AMIs, VMDKs)
- **Vagrant:** Development environment boxes

Immutable vs Mutable

```
Mutable:
Server → Configure → Update → Patch

Immutable:
Image → Deploy → Replace → Deploy New Image
```

Provisioning Tools

Provisioning Tools Purpose

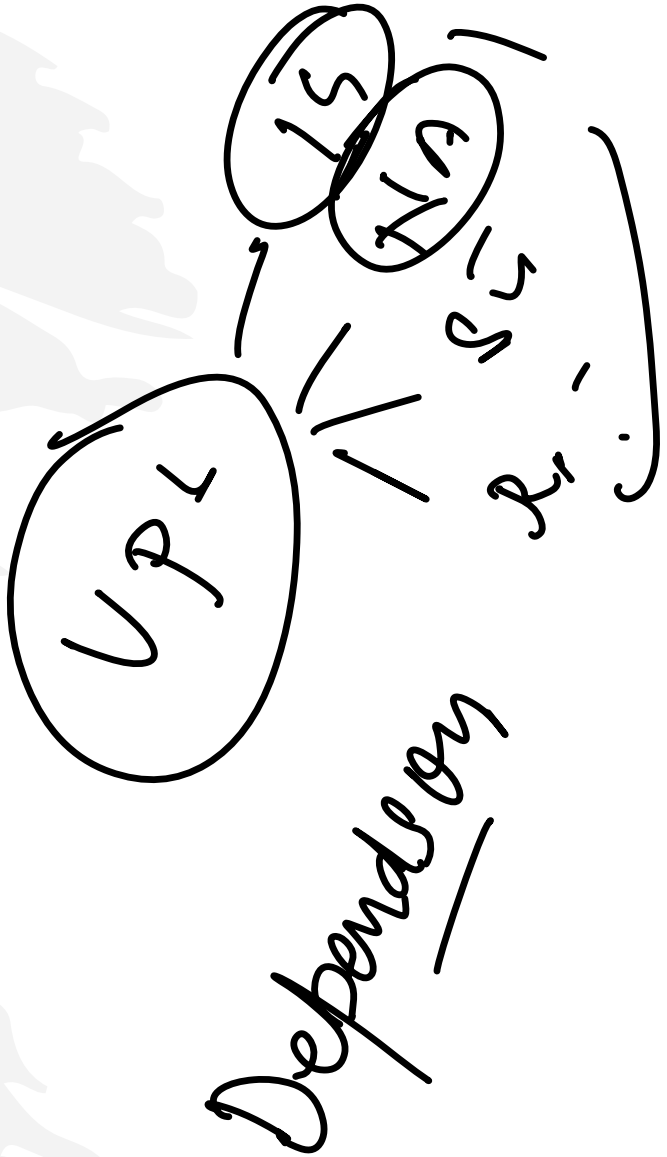
Deploy immutable infrastructure resources

What They Provision

- **Servers** - EC2, VMs, Containers
- **Databases** - RDS, DynamoDB, MongoDB
- **Network Components** - VPCs, Subnets, Load Balancers
- **Security** - IAM, Security Groups, Policies

Multi-Provider Support

- **Cloud Providers** - AWS, Azure, GCP
- **On-Premises** - VMware, OpenStack
- **SaaS** - GitHub, DataDog, Auth0



0

Types of IAC Approaches

1. Declarative Approach (What to achieve)

What is it?

You **describe the desired end state** of your infrastructure, and the tool figures out how to achieve it.

Example (Terraform):

```
resource "aws_instance" "web" {
  ami           = "ami-0c55b159cbfafe1f0"
  instance_type = "t2.micro"
}
```

You're saying: "I want this instance to exist"—Terraform figures out how to create/update/destroy it.

Tools:

- **Terraform**
- **AWS CloudFormation**
- **Bicep**
- **Puppet**

Pros:

- Easier to understand and audit
- Idempotent (running multiple times gives same result)
- Better suited for version control and review
- Tools handle the *how*

Cons:

- Less flexible for conditional logic or loops (unless tool supports it)
- Sometimes harder to debug what changes will be made

2. Imperative Approach (How to do it)

What is it?

You **explicitly define the steps** the system must follow to reach the desired state.
You **write steps** explicitly, like a shell script

Tools: Ansible (hybrid), shell scripts

```
aws s3api create-bucket –bucket my-bucket
aws s3api put-bucket-acl –bucket my-bucket –acl private
```

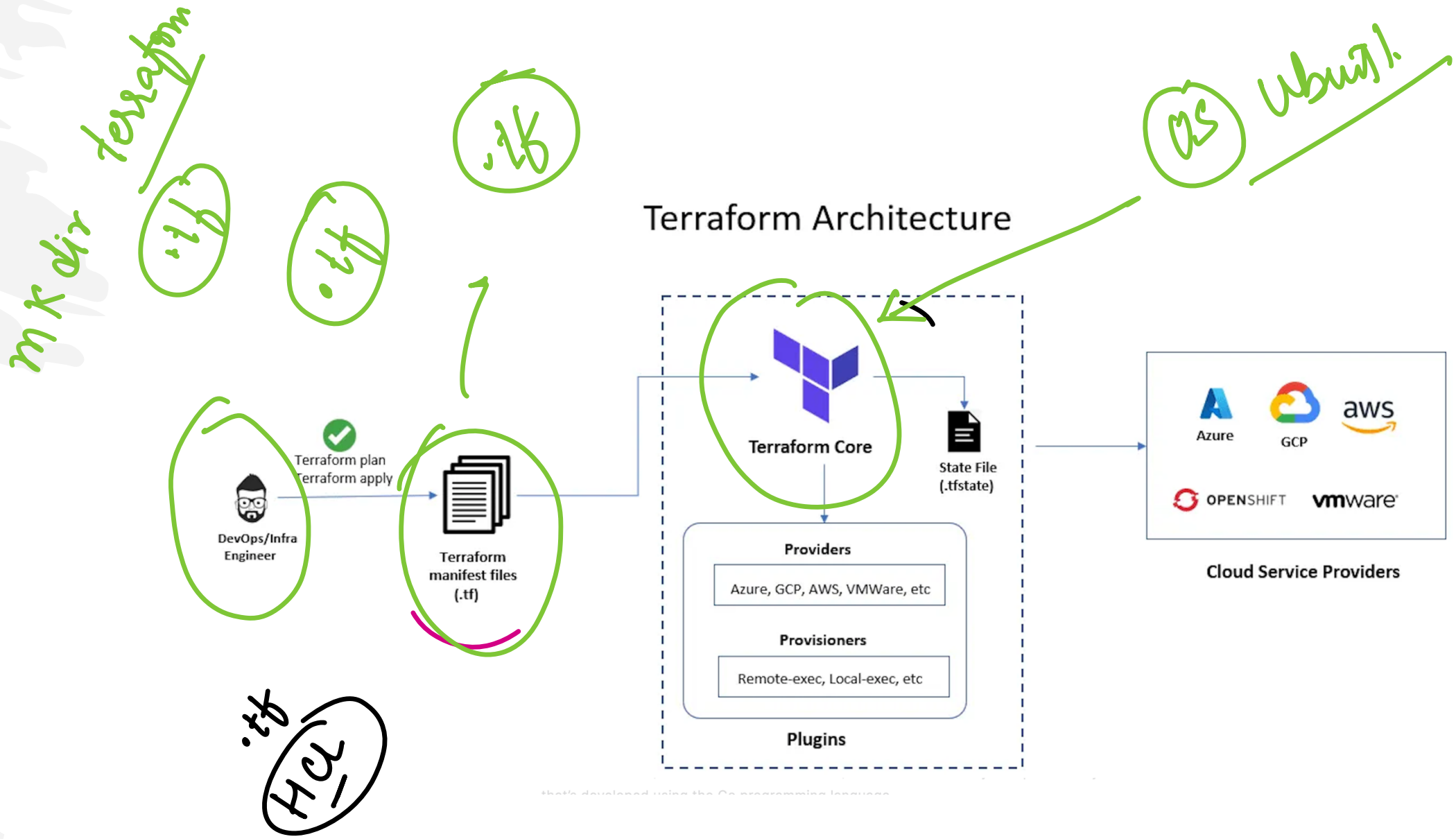
You're telling the system:
“Create this bucket by running these exact commands.”

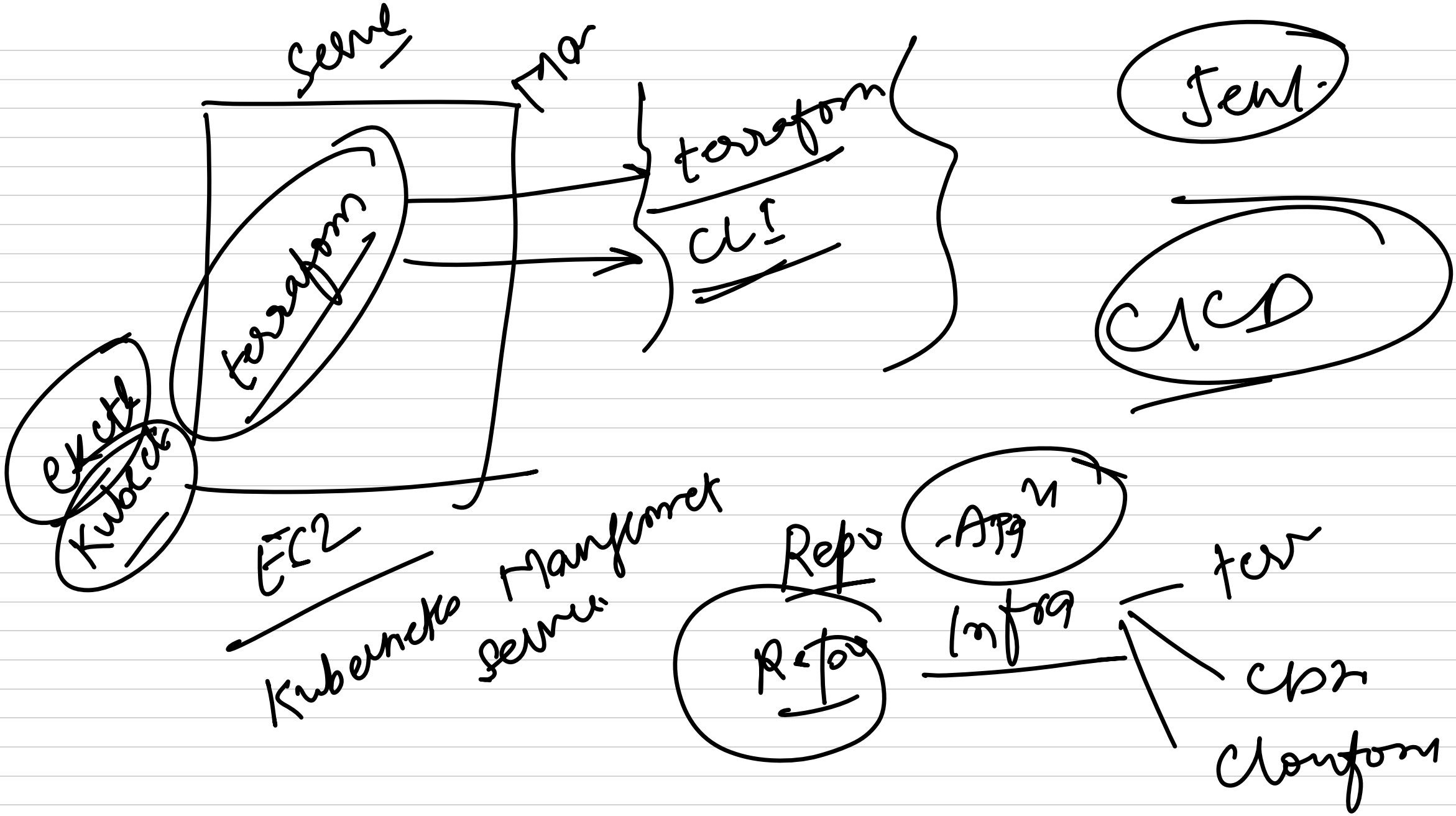
Pros:

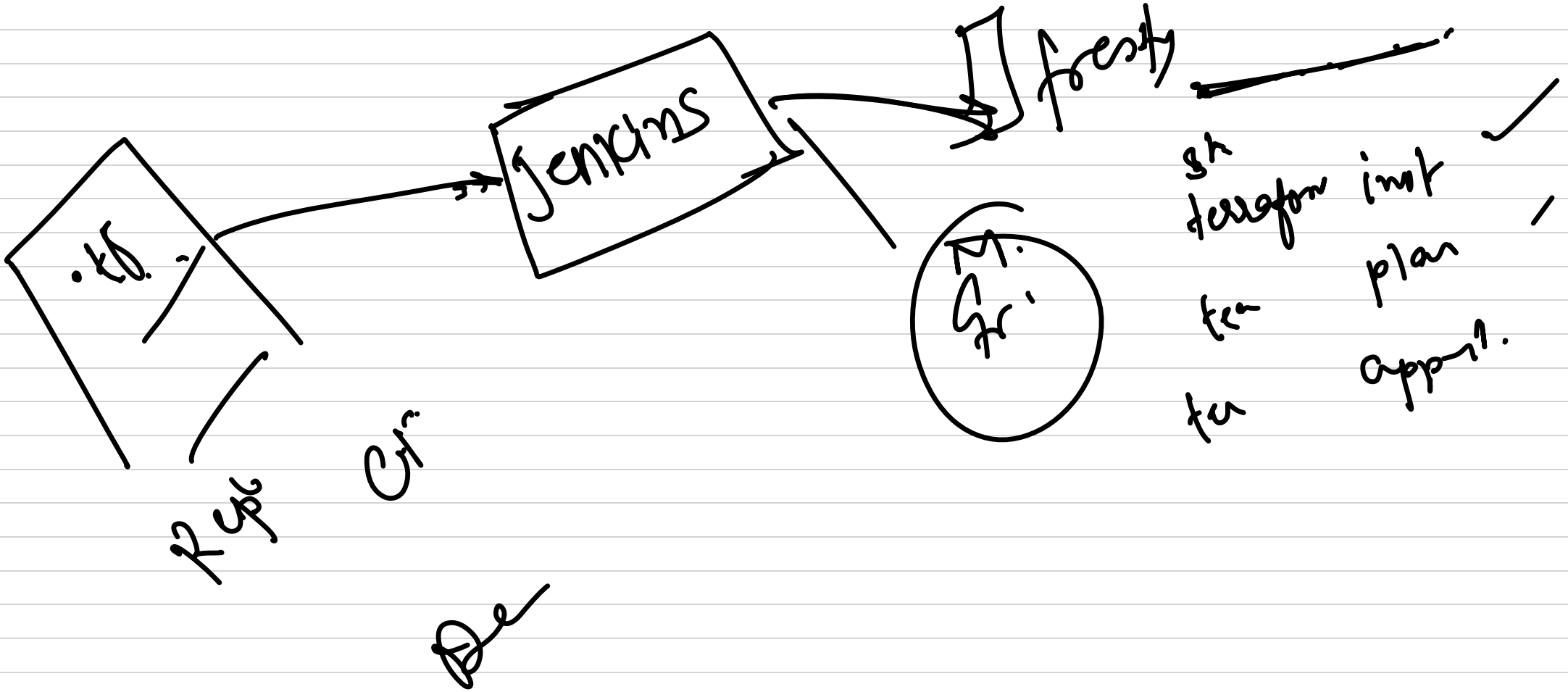
- Fine-grained control over execution
- Easier for those with scripting background
- Can include loops, conditions, and complex logic

Cons:

- Harder to maintain at scale
- Not inherently idempotent unless written carefully
- Less clear "end state" – focuses on *how*, not *what*


Terraform Architecture
DevOps/Infra Engineer
Terraform plan
Terraform apply
Terraform manifest files (.tf)
Terraform Core
State File (.tfstate)
Providers
Azure, GCP, AWS, VMWare, etc
Provisioners
Remote-exec, Local-exec, etc
Plugins
Azure
GCP
aws
OPENSHIFT
vmware
Cloud Service Providers


server
Terraform
terraform
CLI
Jenkins
CICD
eksctl
kubectl
EC2
Kubernetes management server
Repo
Repo
App
Infra
tfvars
eks
cloudform

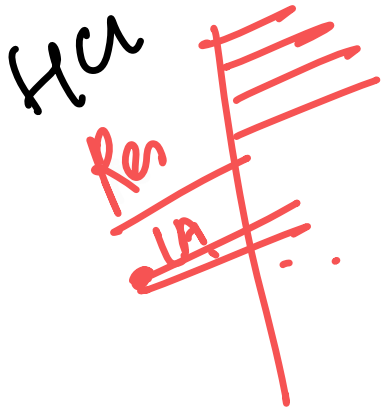

.tf
jenkins
Sr.
gh
terraform init
plan
apply.

HashiCorp Configuration Language

main.tf

```
resource "aws_instance" "webserver" {
  ami           = "ami-0edab43b6fa892279"
  instance_type = "t2.micro"
}

resource "aws_s3_bucket" "finance" {
    bucket = "finanace-21092020"
    tags   = {
        Description = "Finance and Payroll"
    }
}
 resource "aws_iam_user" "admin-user" {
     name = "lucy"
     tags = {
       Description = "Team Leader"
     }
 }
```



Declarative

for init

main.tf

```
resource "aws_instance" "webserver" {
  ami           = "ami-0edab43b6fa892279"
  instance_type = "t2.micro"
}

resource "aws_s3_bucket" "finance" {
    bucket = "finanace-21092020"
    tags   = {
        Description = "Finance and Payroll"
    }
}
 resource "aws_iam_user" "admin-user" {
     name = "lucy"
     tags = {
       Description = "Team Leader"
     }
 }
```

Init ✓

Plan ✓

Apply

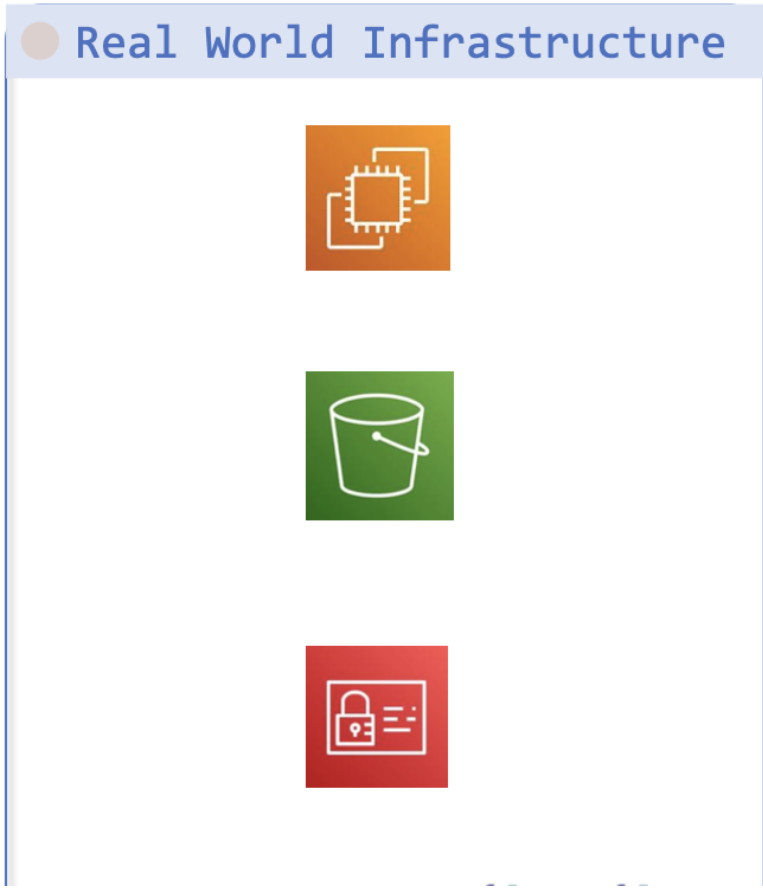
Real World Infrastructure



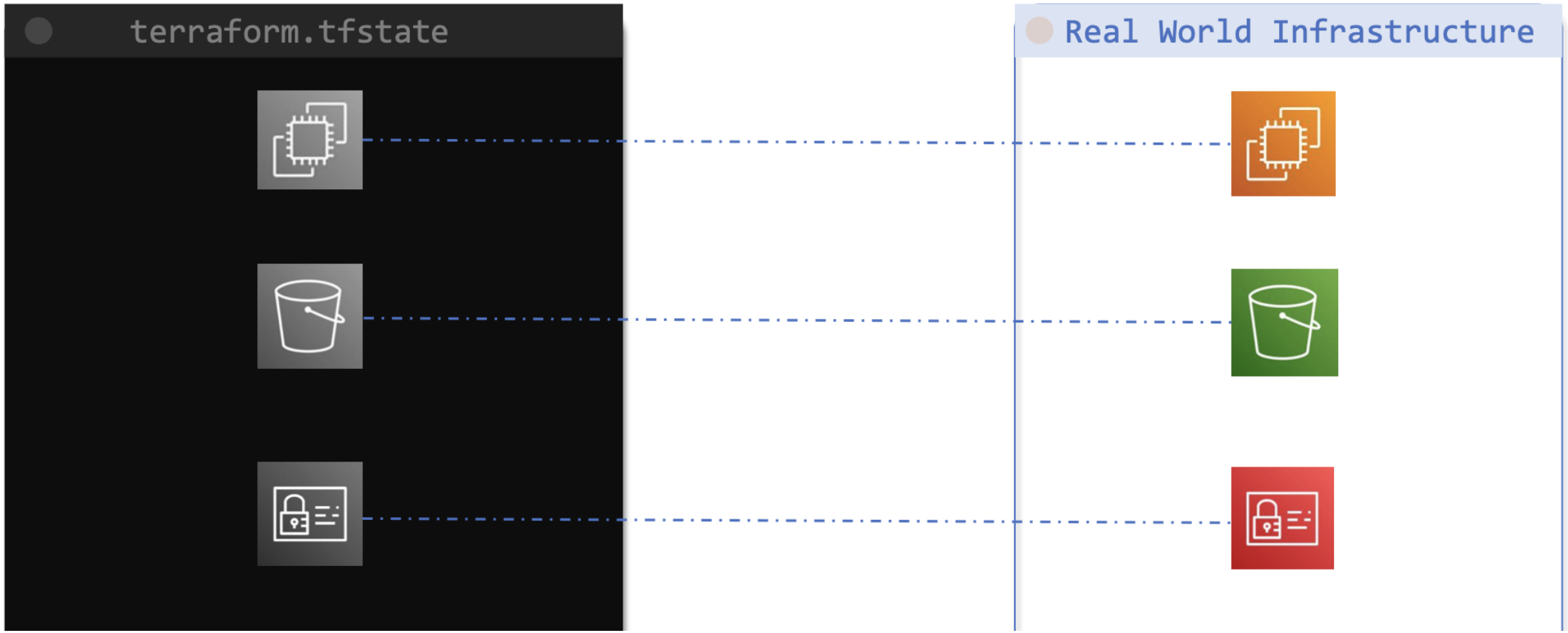




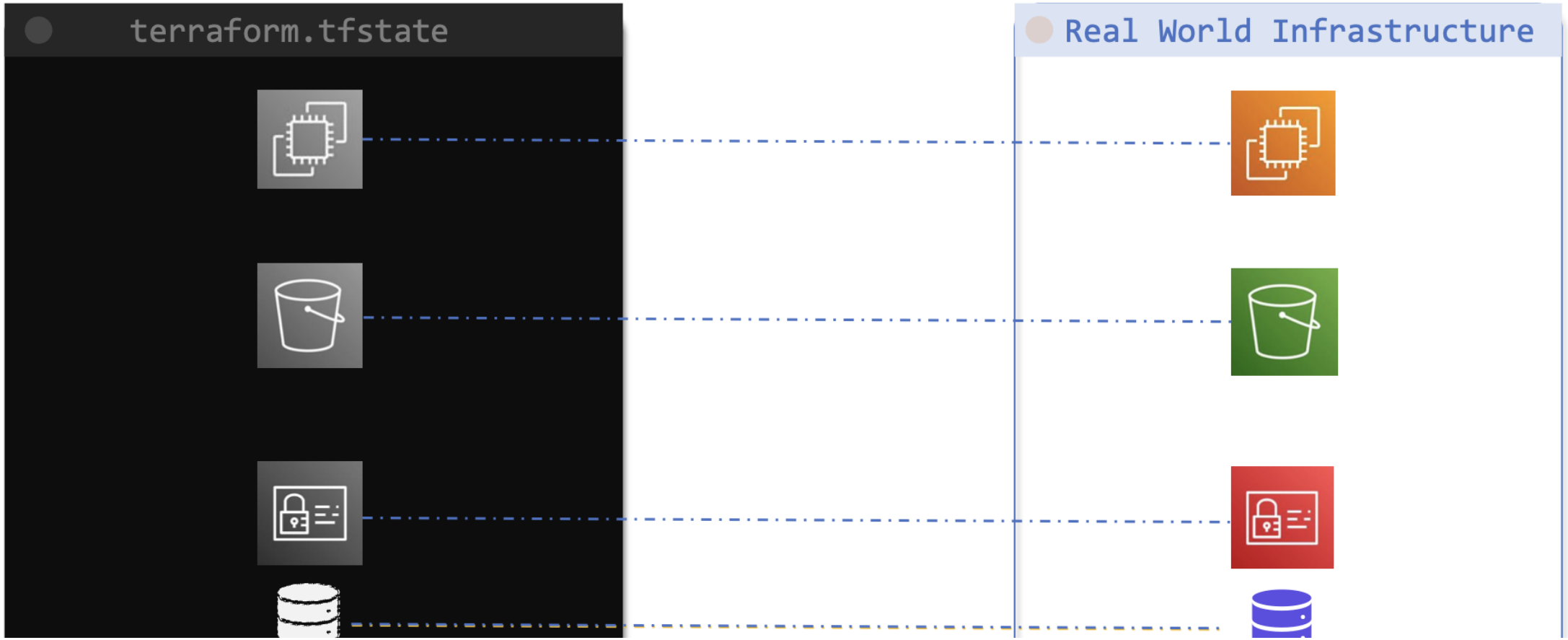
Resource



Terraform State



Terraform Import





Terraform Licensing & Pricing

Understanding the New Business Source License (BSL 1.1)

Key Change: August 10, 2023
From Open Source (MPL 2.0) → Source-Available (BSL 1.1)

License Evolution Timeline

Before Aug 2023
Mozilla Public License v2.0
Fully Open Source

→

Aug 10, 2023
Business Source License v1.1
Source-Available

✅ MPL 2.0 (Old)

- Fully open source
- No commercial restrictions
- Could create competing products
- Complete freedom of use

⚠️ BSL 1.1 (Current)

- Source-available (not open source)
- Commercial restrictions apply
- Cannot compete with HashiCorp
- Free for internal use only

🔑 Key Licensing Points

✅ What You CAN Do (Free)

- **Internal Use:** Manage your own infrastructure
- **Personal Projects:** Individual learning and development
- **Company Projects:** Internal infrastructure management
- **View & Modify:** Source code remains accessible

❌ What You CANNOT Do

- **Commercial Competition:** Create managed Terraform platforms
- **SaaS Offerings:** Sell Terraform-as-a-Service
- **Direct Competition:** Compete with HashiCorp's products
- **Redistribution:** Package and sell Terraform commercially

💰 Terraform Editions & Pricing

Community Edition

FREE

- ✓ Command-line interface (CLI)
- ✓ Core Terraform functionality
- ✓ Write, plan, and apply changes
- ✓ Local state management
- ✓ Provider ecosystem access

Terraform Cloud

$20+/user

- ✓ Remote state management
- ✓ Team collaboration
- ✓ Policy as code
- ✓ Cost estimation
- ✓ VCS integration
- ✓ Concurrent runs

Terraform Enterprise

Custom

- ✓ Self-hosted deployment
- ✓ Advanced security features
- ✓ Compliance controls
- ✓ Audit logging
- ✓ SSO integration
- ✓ Priority support

🌟 OpenTofu: The Open Source Alternative

🚀 OpenTofu

Community Response to Terraform's License Change

✅ Benefits

- Truly open source (MPL 2.0)
- Linux Foundation managed
- Terraform-compatible
- No commercial restrictions
- Community-driven development

🎯 Use Cases

- Commercial Terraform alternatives
- SaaS platform development
- Enterprise with licensing concerns
- Open source compliance requirements
- Long-term vendor independence

A company has a legacy application. A DevOps engineer needs to automate the process of building the deployable artifact for the legacy application. The solution must store the deployable artifact in an existing Amazon S3 bucket for future deployments to reference.

Which solution will meet these requirements in the MOST operationally efficient way?

A. Create a custom Docker image that contains all the dependencies for the legacy application. Store the custom Docker image in a new Amazon Elastic Container Registry (Amazon ECR) repository. Configure a new AWS CodeBuild project to use the custom Docker image to build the deployable artifact and to save the artifact to the S3 bucket.

B. Launch a new Amazon EC2 instance. Install all the dependencies for the legacy application on the EC2 instance. Use the EC2 instance to build the deployable artifact and to save the artifact to the S3 bucket.

C. Create a custom EC2 Image Builder image. Install all the dependencies for the legacy application on the image. Launch a new Amazon EC2 instance from the image. Use the new EC2 instance to build the deployable artifact and to save the artifact to the S3 bucket.

D. Create an Amazon Elastic Kubernetes Service (Amazon EKS) cluster with an AWS Fargate profile that runs in multiple Availability Zones. Create a custom Docker image that contains all the dependencies for the legacy application. Store the custom Docker image in a new Amazon Elastic Container Registry (Amazon ECR) repository. Use the custom Docker image inside the EKS cluster to build the deployable artifact and to save the artifact to the S3 bucket.

A company has an application that runs on Amazon EC2 instances. The company uses an AWS CodePipeline pipeline to deploy the application into multiple AWS Regions. The pipeline is configured with a stage for each Region. Each stage contains an AWS CloudFormation action for each Region.

When the pipeline deploys the application to a Region, the company wants to confirm that the application is in a healthy state before the pipeline moves on to the next Region. Amazon Route 53 record sets are configured for the application in each Region. A DevOps engineer creates a Route 53 health check that is based on an Amazon CloudWatch alarm for each Region where the application is deployed.

What should the DevOps engineer do next to meet the requirements?

A. Create an AWS Step Functions workflow to check the state of the CloudWatch alarm. Configure the Step Functions workflow to exit with an error if the alarm is in the ALARM state. Create a new stage in the pipeline between each Region deployment stage. In each new stage, include an action to invoke the Step Functions workflow.

B. Configure an AWS CodeDeploy application to deploy a CloudFormation template with automatic rollback. Configure the CloudWatch alarm as the instance health check for the CodeDeploy application. Remove the CloudFormation actions from the pipeline. Create a CodeDeploy action in the pipeline stage for each Region.

C. Create a new pipeline stage for each Region where the application is deployed. Configure a CloudWatch alarm action for the new stage to check the state of the CloudWatch alarm and to exit with an error if the alarm is in the ALARM state

D. Configure the CloudWatch agent on the EC2 instances to report the application status to the Route 53 health check. Create a new pipeline stage for each Region where the application is deployed. Configure a CloudWatch alarm action to exit with an error if the CloudWatch alarm is in the ALARM state.

A company plans to use Amazon CloudWatch to monitor its Amazon EC2 instances. The company needs to stop EC2 instances when the average of the NetworkPacketsIn metric is less than 5 for at least 3 hours in a 12-hour time window. The company must evaluate the metric every hour. The EC2 instances must continue to run if there is missing data for the NetworkPacketsIn metric during the evaluation period.

A DevOps engineer creates a CloudWatch alarm for the NetworkPacketsIn metric. The DevOps engineer configures a threshold value of 5 and an evaluation period of 1 hour.

Which set of additional actions should the DevOps engineer take to meet these requirements?

A. Configure the Datapoints to Alarm value to be 3 out of 12. Configure the alarm to treat missing data as breaching the threshold. Add an AWS Systems Manager action to stop the instance when the alarm enters the ALARM state.

B. Configure the Datapoints to Alarm value to be 3 out of 12. Configure the alarm to treat missing data as not breaching the threshold. Add an EC2 action to stop the instance when the alarm enters the ALARM state.

C. Configure the Datapoints to Alarm value to be 9 out of 12. Configure the alarm to treat missing data as breaching the threshold. Add an EC2 action to stop the instance when the alarm enters the ALARM state.

D. Configure the Datapoints to Alarm value to be 9 out of 12. Configure the alarm to treat missing data as not breaching the threshold. Add an AWS Systems Manager action to stop the instance when the alarm enters the ALARM state.

A company manages 500 AWS accounts that are in an organization in AWS Organizations. The company discovers many unattached Amazon Elastic Block Store (Amazon EBS) volumes in all the accounts. The company wants to automatically tag the unattached EBS volumes for investigation.

A DevOps engineer needs to deploy an AWS Lambda function to all the AWS accounts. The Lambda function must run every 30 minutes to tag all the EBS volumes that have been unattached for a period of 7 days or more.

Which solution will meet these requirements in the MOST operationally efficient manner?

A. Configure a delegated administrator account for the organization. Create an AWS CloudFormation template that contains the Lambda function. Use CloudFormation StackSets to deploy the CloudFormation template from the delegated administrator account to all the member accounts in the organization. Create an Amazon EventBridge event bus in the delegated administrator account to invoke the Lambda function in each member account every 30 minutes.

B. Create a cross-account IAM role in the organization's member accounts. Attach the AWSLambda_FullAccess policy and the AWSCloudFormationFullAccess policy to the role. Create an AWS CloudFormation template that contains the Lambda function and an Amazon EventBridge scheduled rule to invoke the Lambda function every 30 minutes. Create a custom script in the organization’s management account that assumes the role and deploys the CloudFormation template to the member accounts.

C. Configure a delegated administrator account for the organization. Create an AWS CloudFormation template that contains the Lambda function and an Amazon EventBridge scheduled rule to invoke the Lambda function every 30 minutes. Use CloudFormation StackSets to deploy the CloudFormation template from the delegated administrator account to all the member accounts in the organization

D. Create a cross-account IAM role in the organization's member accounts. Attach the AmazonS3FullAccess policy and the AWSCodeDeployDeployerAccess policy to the role. Use AWS CodeDeploy to assume the role to deploy the Lambda function from the organization's management account. Configure an Amazon EventBridge scheduled rule in the member accounts to invoke the Lambda function every 30 minutes.

A company's production environment uses an AWS CodeDeploy blue/green deployment to deploy an application. The deployment incudes Amazon EC2 Auto Scaling groups that launch instances that run Amazon Linux 2.

A working appspec.yml file exists in the code repository and contains the following text:

```
version: 0.0
os: linux
files:
  - source: /
    destination: /var/www/html/application
```

A DevOps engineer needs to ensure that a script downloads and installs a license file onto the instances before the replacement instances start to handle request traffic.
The DevOps engineer adds a hooks section to the appspec.yml file.

Which hook should the DevOps engineer use to run the script that downloads and installs the license file?

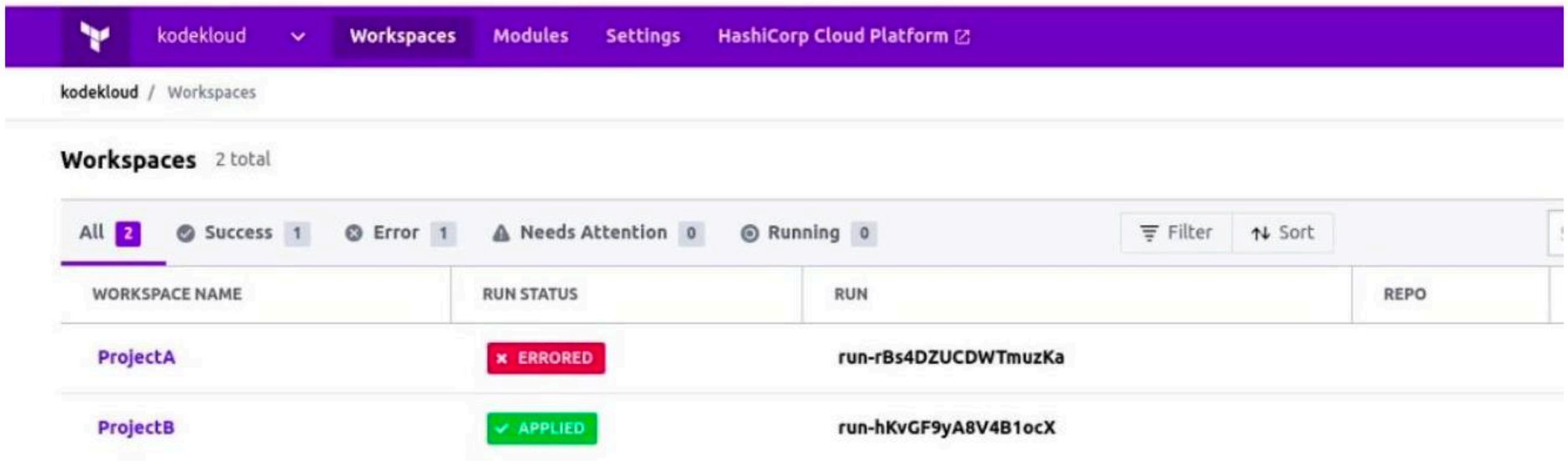
A. AfterBlockTraffic

B. BeforeBlockTraffic

C. BeforeInstall

D. DownloadBundle

Terraform Cloud and Terraform Enterprise



```
>_

$ wget https://releases.hashicorp.com/terraform/0.13.0/terraform_0.13.0_linux_amd64.zip
$ unzip terraform_0.13.0_linux_amd64.zip
$ mv terraform /usr/local/bin
$ terraform version
  Terraform v0.13.0
```



macOS
64-bit



FreeBSD
32-bit | 64-bit | Arm



Linux
32-bit | 64-bit | Arm



OpenBSD
32-bit | 64-bit

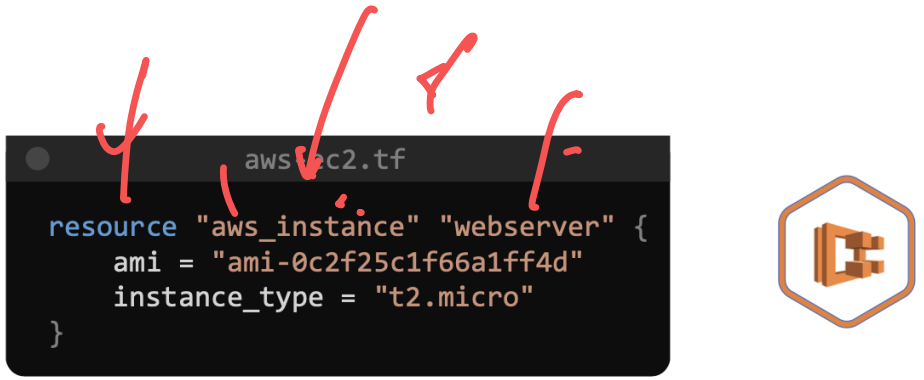


Solaris
64-bit



Windows
32-bit | 64-bit


local-provider
Block Name
Resource Type
Resource Name
FILENAME
CONTENT
local.tf
resource "local_file" "pet" {
filename = "/root/pets.txt"
content = "We love pets!"
}
Arguments


aws_ec2.tf
resource "aws_instance" "webserver" {
ami = "ami-0c2f25c1f66a1ff4d"
instance_type = "t2.micro"
}


aws-s3.tf
resource "aws_s3_bucket" "data" {
bucket = "webserver-bucket-org-2207"
acl = "private"
}


local.tf
resource "local_file" "pet" {
filename = "/root/pets.txt"
content = "We love pets!"
}


Init
Plan
Apply

```
$ terraform init
Initializing the backend...

Initializing provider plugins...
- Finding latest version of hashicorp/local...
- Installing hashicorp/local v1.4.0...
- Installed hashicorp/local v1.4.0 (signed by HashiCorp)

The following providers do not have any version constraints in configuration,
so the latest version was installed.

To prevent automatic upgrades to new major versions that may contain breaking
changes, we recommend adding version constraints in a required_providers block
in your configuration, with the constraint strings suggested below.

* hashicorp/local: version = "~> 1.4.0"

Terraform has been successfully initialized!
```

```
$ terraform plan
Refreshing Terraform state in-memory prior to plan...
The refreshed state will be used to calculate this plan, but will not be
persisted to local or remote state storage.


------------------------------------------------------------------------

An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.pet will be created
  + resource "local_file" "pet" {
      + content              = "We love pets!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "/root/pets.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

------------------------------------------------------------------------

Note: You didn't specify an "-out" parameter to save this plan, so
Terraform
can't guarantee that exactly these actions will be performed if
"terraform apply" is subsequently run.
```

```
$ terraform apply
An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # local_file.pet will be created
  + resource "local_file" "pet" {
      + content              = "We love pets!"
      + directory_permission = "0777"
      + file_permission      = "0777"
      + filename             = "/root/pets.txt"
      + id                   = (known after apply)
    }

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
local_file.new_file: Creating...
local_file.new_file: Creation complete after 0s
[id=521c5c732c78cb42cc9513ecc7c0638c4a115b55]
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

$ cat /root/pets.txt
We love pets!
```

```
$ terraform show
# local_file.pet:
resource "local_file" “pet" {
    content              = "We love pets!"
    directory_permission = "0777"
    file_permission      = "0777"
    filename             = "/root/pets.txt"
    id                   = "cba595b7d9f94ba1107a46f3f731912d95fb3d2c"
}
```